

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №3

Выполнил:

Зотеев Максим

БР1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Выполнить контейнеризацию реализованного ранее бэкенд-приложения средствами Docker. Для каждого сервиса должен быть подготовлен собственный Dockerfile, для всей системы — общий docker-compose.yml, обеспечивающий сетевое взаимодействие между сервисами и зависимостями (база данных, брокер сообщений). Стенд должен подниматься одной командой и быть готов к локальной проверке.

Ход работы

1. Что упаковывается в контейнеры

Контейнеризуется микросервисная система сервиса аренды недвижимости, реализованная в LP2. В её состав входят пять прикладных сервисов на Node.js + TypeScript: api-gateway (внешний шлюз), identity-service (пользователи и аутентификация), property-service (объекты и справочники), rental-service (сделки) и messaging-service (чат и системные уведомления). К ним добавляются два инфраструктурных компонента: PostgreSQL 16 в общем контейнере с отдельной БД на сервис и RabbitMQ 3.13 с management-плагином для асинхронного межсервисного взаимодействия из Д35. Артефакты Dockerfile и docker-compose.yml физически лежат в каталоге labs/lab2/.

2. Dockerfile для каждого сервиса

Все пять прикладных сервисов используют одинаковый шаблон Dockerfile на базе официального образа node:20-alpine, выбранного за минимальный размер. Сборка многоэтапная в логике, но в одном слое: рабочая директория /app, копирование package.json + tsconfig.json, npm install, копирование исходников из src/, сборка через npm run build (запускает tsc), запуск runtime командой node dist/index.js. В каждом сервисе объявлен EXPOSE на свой внутренний порт (3001 для identity, 3002 для property, 3003 для rental, 3004 для messaging, 8080 для api-gateway). Использование alpine-варианта Node даёт итоговые образы порядка 250–300 МБ на сервис.

3. Общий docker-compose.yml

Единый docker-compose.yml описывает семь сервисов: postgres, rabbitmq, identity-service, property-service, rental-service, messaging-service и api-gateway. Каждый прикладной сервис собирается локально (build: ./services/<name>), пробрасывает наружу свой порт для удобства отладки и получает конфигурацию через переменные окружения. Чувствительные значения (JWT_SECRET, INTERNAL_SERVICE_TOKEN) подтягиваются из .env с дефолтами на случай его отсутствия. Для постоянного хранения данных PostgreSQL объявлен именованный том pgdata.

4. Сетевое взаимодействие между сервисами

Docker Compose автоматически создаёт изолированную bridge-сеть lab2_default, в которую попадают все контейнеры стенда. Внутри этой сети сервисы обращаются друг к другу по DNS-именам, совпадающим с именами сервисов из compose: rental-service знает identity-service как http://identity-service:3001 и property-service как http://property-

service:3002, messaging-service — rental-service как `http://rental-service:3003`, api-gateway — все четыре прикладных сервиса. Конкретные URL передаются переменными окружения `IDENTITY_SERVICE_URL`, `PROPERTY_SERVICE_URL`, `RENTAL_SERVICE_URL`, `MESSAGING_SERVICE_URL`. RabbitMQ доступен сервисам по `amqp://rental:rental@rabbitmq:5672`. Снаружи системе наружу торчит только api-gateway на порту 8080; внутренний api (`/api/v1/internal/*`) физически доступен по портам 3001–3004, но на уровне маршрутизации шлюза отсечён `pathFilter`ом и не пробрасывается.

5. Healthcheck'и и порядок старта

Корректный порядок поднятия контейнеров обеспечивается комбинацией `depends_on` и `healthcheck`ов. У `postgres` `healthcheck` выполняет `pg_isready -U rental`, у `rabbitmq` — `rabbitmq-diagnostics -q ping`. Прикладные сервисы объявляют `depends_on` с условием `service_healthy` на инфраструктуру, что гарантирует, что они не стартуют до готовности БД и брокера. Зависимые прикладные сервисы (`rental`, `messaging`, `api-gateway`) дополнительно ждут условие `service_started` на сервисах, которые они вызывают. На случай гонок при первом подключении к RabbitMQ в коде каждого сервиса добавлены `retry`-попытки (10 итераций по 2 секунды).

6. Запуск и проверка

Полный жизненный цикл стенда сводится к трём командам: `cp .env.example .env`, `docker compose up --build` для первого запуска (или `docker compose up -d` для повторного), `docker compose down -v` для полной очистки вместе с томом БД. После старта `health`-эндпоинты доступны на `http://localhost:8080/health` (gateway) и `http://localhost:3001–3004/health` (внутренние сервисы), web-консоль RabbitMQ — на `http://localhost:15672`. Корректность сетевого взаимодействия и работы всей системы целиком подтверждена прогоном Postman-коллекции через `newman`: 62 теста, 127 `assert`ов, время прогона около 3 секунд, ошибок нет.

Заключение

В рамках ЛР3 выполнена контейнеризация микросервисного бэкенд-приложения, реализованного в ЛР2 и расширенного асинхронной шиной в Д35. Подготовлены отдельные `Dockerfile` для всех пяти прикладных сервисов на базе `node:20-alpine`, описан общий `docker-compose.yml` с инфраструктурой (PostgreSQL и RabbitMQ), настроено сетевое взаимодействие через автоматическую bridge-сеть Compose с обращением сервисов друг к другу по `hostname`-именам, заданы `healthcheck`'и и порядок старта через `depends_on`. Стенд поднимается одной командой `docker compose up --build`, корректность сетевого взаимодействия подтверждена прогоном автотестов через `postman`